The Ford-Fulkerson Algorithm

Maximum Flow in Network Graphs

Why Maximum Flow?

Imagine a network of **drainage ditches** on a farm. Water flows from a pond through junctions to a creek, and each ditch has a limited capacity. *What is the maximum amount of water we can drain per second?*

This is the **Maximum Flow Problem** — it appears in network bandwidth, airline scheduling, bipartite matching, and more.

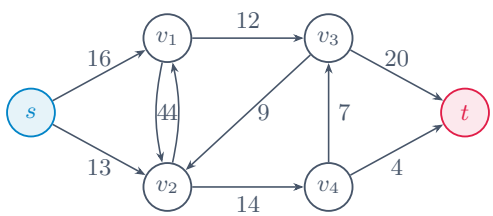
Flow Network

Definition

A **flow network** $G = (V, E)$ has a **source** s, **sink** t, and capacity $c(u, v) \geq 0$ on each edge. A **flow** f assigns a value to each edge satisfying:

1. **Capacity**: $0 \leq f(u, v) \leq c(u, v)$
2. **Conservation**: $\sum_v f(v, u) = \sum_v f(u, v)$ for $u \neq s, t$

The **flow value** is $|f| = \sum_v f(s, v) - \sum_v f(v, s)$ (net outflow from source).



Example flow network (CLRS Fig. 26.1)

Residual Network & Augmenting Paths

Residual Capacity

Given flow f, the **residual capacity** of edge (u, v):

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E \\ f(v, u) & \text{if } (v, u) \in E \\ 0 & \text{otherwise} \end{cases}$$

The **residual network** $G_f = (V, E_f)$ with $E_f = \{(u, v) : c_f(u, v) > 0\}$.

Why back-edges?

Back-edges let the algorithm *undo* earlier decisions. Without them, a greedy approach can get stuck at a sub-optimal flow. Back-edges are the key insight that makes Ford-Fulkerson *optimal*.

An **augmenting path** is any $s \to t$ path in G_f with $c_f > 0$ on every edge. Its **bottleneck**: $c_f(p) = \min\{c_f(u, v) : (u, v) \in p\}$.

The Ford-Fulkerson Method

Ford-Fulkerson Algorithm (with BFS)

1. **Initialize** all edge flows to zero: $f(u, v) = 0$ for all $(u, v) \in E$
2. **While** there exists an augmenting path p from s to t in G_f:
   (a) Find p using **BFS** on G_f (guarantees shortest path)
   (b) Compute bottleneck: $c_f(p) = \min\{c_f(u, v) : (u, v) \in p\}$
   (c) Update flow along p: forward edge: $f(u, v) \mathrel{+}= c_f(p)$, back-edge: $f(v, u) \mathrel{-}= c_f(p)$
3. **Return** f — maximum flow reached!

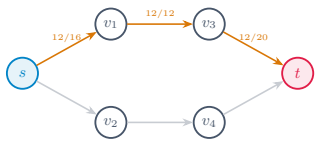
Complexity: $O(VE^2)$ using BFS.

Step-by-Step Example

Starting from the graph on page 1, all edge flows begin at 0.

Iteration 1 +12

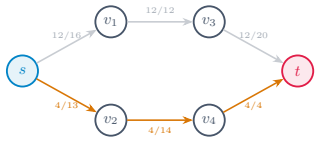
BFS path: $s \to v_1 \to v_3 \to t$
Bottleneck: $\min(16, 12, 20) = \mathbf{12}$



$|f| = 0 + 12 = \mathbf{12}$

Iteration 2 +4

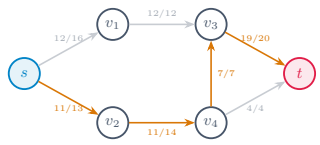
BFS path: $s \to v_2 \to v_4 \to t$
Bottleneck: $\min(13, 14, 4) = \mathbf{4}$



$|f| = 12 + 4 = \mathbf{16}$

Iteration 3 +7

BFS path: $s \to v_2 \to v_4 \to v_3 \to t$
Bottleneck: $\min(9, 10, 7, 8) = \mathbf{7}$



$|f| = 16 + 7 = \mathbf{23}$

BFS finds no more augmenting paths.
After 3 iterations, maximum flow $|f^*| = \mathbf{23}$.

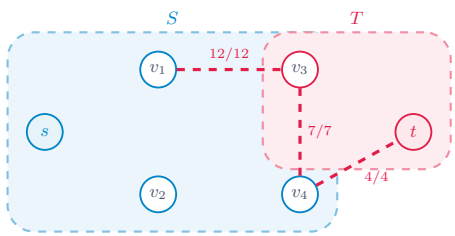
Cuts and the Max-Flow Min-Cut Theorem

Definition: Cut

A **cut** (S, T) partitions vertices into $S \ni s$ and $T \ni t$.

Capacity: $c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$

Cut	S	$c(S, T)$
Cut 1	$\{s\}$	$16 + 13 = \mathbf{29}$
Cut 2	$\{s, v_1, v_2\}$	$12 + 14 = \mathbf{26}$
Min Cut	$\{s, v_1, v_2, v_4\}$	$12 + 7 + 4 = \mathbf{23}$



Min cut capacity: $12 + 7 + 4 = 23 = |f^*|$

Max-Flow Min-Cut Theorem

The following three statements are **equivalent**:

1. f is a **maximum flow** in G
2. The residual network G_f contains **no augmenting path**
3. There exists a cut (S, T) such that $|f| = c(S, T)$

Proof sketch: When BFS fails to reach t in G_f, define $S = \{\text{nodes reachable from } s\}$ and $T = V \setminus S$. Every $S \to T$ edge must be saturated (otherwise BFS would cross it), so $|f| = c(S, T)$. Since $|f| \le c(S', T')$ for *any* cut, this flow is maximum and this cut is minimum.

Key Takeaways

1. **Residual Network** G_f shows remaining capacity + back-edges for undo
2. **Augmenting Path**: $s \to t$ path in G_f; use **BFS** for $O(VE^2)$ guarantee
3. **Max-Flow = Min-Cut**: algorithm stops when no path exists; flow equals min cut